

AI 비전검수 시스템

상태머신 · 노드 책임 · 정상 운전 · 오류 복구 워크플로우

MasterNode · ControlNode · VisionNode · LogNode

상층/하층 컨베이어 · 양면 정지 촬영 · 실물 액추에이터 분류

드롭퍼형 화장품 캡 $\varnothing 40 \times h20$ · Hikrobot MV-CS050-10GC $\times 4$ · Arduino Mega 2560 + TB6600

설계 기준 요약

항목	확정 내용
검사 방식	상층에서 한 면 촬영 → FLIPPING → 하층에서 반대 면 촬영 → Sensor 3 판정 잠금 → 액추에이터 분류
정합성 원본	제품 ID · 활성 FIFO · 최종 판정은 MasterNode 단일 소유. 다른 노드는 별도 FIFO를 만들지 않는다
실물 제어	Arduino Mega 2560 + TB6600. ControlNode가 센서·컨베이어·액추에이터·트리거·조명의 실제 상태를 소유
비전	정지 촬영 + 하드웨어 트리거. Vision은 카메라 ARM과 프레임 수집, 트리거 펄스와 조명 출력은 Control/Mega
오류 철학	제품 단위 문제는 FORCED_NG 후 계속 운전. 제품 추적 무결성을 잃은 경우에만 FAULT_STOP
물리 안전	래칭형 비상정지 스위치가 TB6600 구동전원을 직접 차단. 소프트웨어는 상태 감시와 기록만 담당
정보 문서	상태명·필드명·Enum이 어긋나면 05 데이터 구조·설정값 정의 를 기준으로 맞춘다

1 전체 시스템 흐름

제품 한 개가 투입되어 분류 완료로 제거되기까지의 기준 경로

제품 투입 · Sensor 1 수락

Master가 product_id를 발급하고 활성 FIFO 뒤에 등록한다



상층 정지 촬영 · STATION A

Vision ARMED → Control/Mega가 조명 점등·안정화 후 하드웨어 트리거 발행 → 프레임 수집



상층 재가동 · FLIPPING

촬영 응답 처리 직후 재가동. 제품이 하층으로 이동하며 뒤집힌다



Sensor 2 수락 · 하층 정지 촬영 · STATION B

A/B 추론은 컨베이어 이동과 비동기로 계속 진행된다



Sensor 3 수락 · 판정 잠금

결과가 미완료여도 기다리지 않고 FORCED_NG, PASS/REJECT와 actuator_job_id를 확정한다



액추에이터 분류

예약 route로 통과 감시 → completion_id 생성 → Master ACK



제품 DONE · 영구 저장

FIFO 선두부터 순차 제거. SQLite 레코드와 이미지 파일 보존

촬영이 끝나면 컨베이어는 즉시 재가동하며 추론 완료를 기다리지 않는다. Sensor 3 시점에도 결과가 없으면 라인을 멈추지 않고 해당 제품만 **FORCED_NG**로 고정하고, 뒤늦게 도착한 결과는 **LATE_IGNORED**로 기록만 한다.

2 노드 책임과 상태 소유권

하나의 상태에는 값을 확정할 수 있는 소유 노드가 하나만 존재한다

노드	원본 상태·데이터	핵심 책임
MasterNode	SystemState · pause_reason · 활성 FIFO · ProductContext · 최종 판정 · 세션과 command_epoch	제품 ID 발급, 센서 이벤트와 제품 연결, 공정 조율, FAULT_STOP 결정
ControlNode	컨베이어·센서·액추에이터 실제 상태 · Mega 링크 · E-stop · 트리거와 조명 출력	PUL/DIR/ENA 제어, 스텝 추적, 물리 이벤트 생성, 로컬 안전 정지
VisionNode	카메라 연결 상태 · CaptureJob · InferenceJob · StationResult	카메라 ARM과 프레임 수집, 이미지 저장, PatchCore 추론, 결과 발행
LogNode	SQLite 레코드 · health_state · SpoolState	제품·장비·비전·오류 이벤트 영구 저장과 조회. 운전 명령 중계 없음

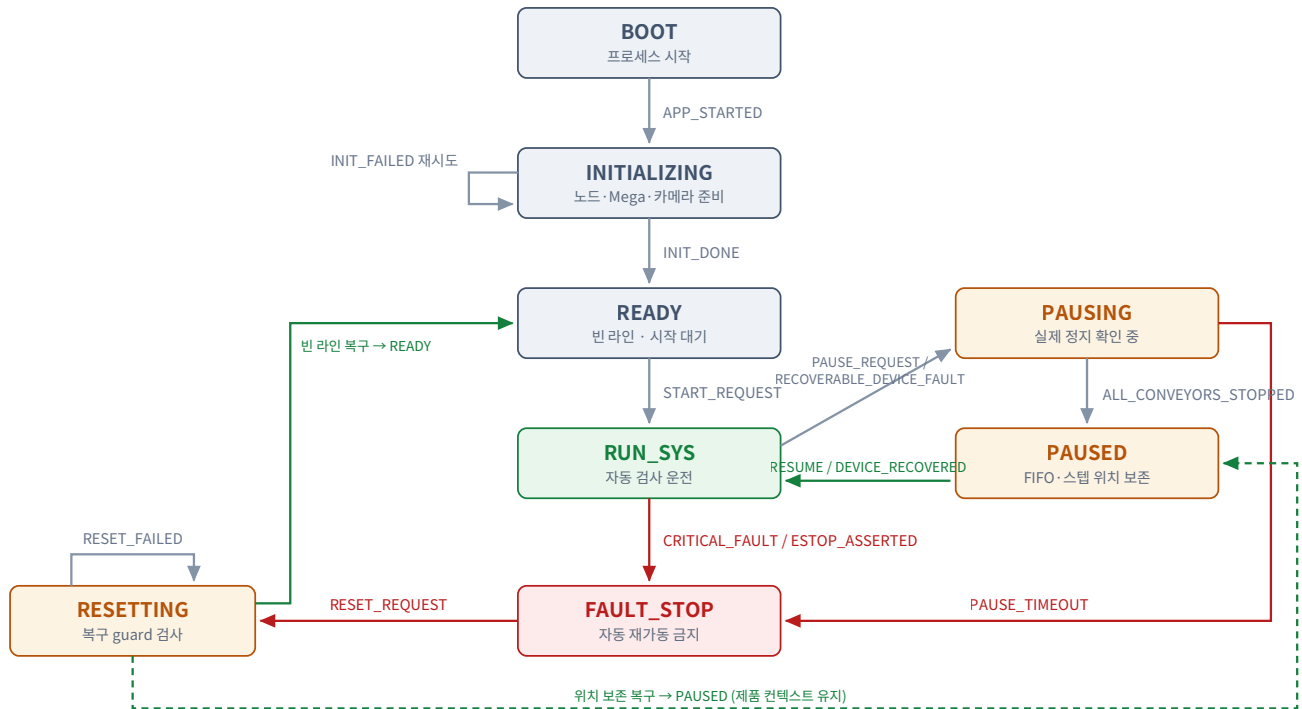
요청 상태와 실제 상태를 분리한다. Master가 **RUN_CONV**를 요청해도 실제 상태를 확정하는 것은 ControlNode뿐이다. 통신 단절이나 안전 상황에서 ControlNode는 Master 응답을 기다리지 않고 즉시 안전 정지할 수 있다.

금지 책임

노드	하지 않는 일
Master	센서 신호 직접 판독, 모터·액추에이터 직접 구동, 카메라 SDK 호출, 로그 파일 직접 기록
Control	제품 ID 생성, 별도 FIFO 운영, 최종 OK/NG 판정, 추론 미완료 제품의 강제 NG 결정
Vision	컨베이어·액추에이터 제어, 트리거 펄스와 조명 채널 직접 출력, 재가동 결정, FAULT_STOP 변경
Log	제품 상태·FIFO 변경, 장비 제어, 판정 변경, FAULT_STOP 결정, 운전 명령 중계

3 전체 시스템 상태머신

MasterNode가 소유하는 운전 FSM. 프로그램 종료 절차는 이 FSM 외부에서 처리한다



회색 = 정상 진행 · 빨강 = 오류 전이 · 초록 = 복구 경로 · RESETTING의 목적지는 복구 정책이 결정한다

주요 전이 대응표

ID	전이	조건 요약
SYS-02·03	INITIALIZING ↔ 재시도	모든 노드 READY와 인터페이스 버전 일치 시 READY. 실패는 안전 정지 상태에서 간격 재시도
SYS-04	READY → RUN_SYS	빈 FIFO와 빈 라인 확인 후 상·하층 실제 RUN_CONV 확인
SYS-05 · 05A	RUN_SYS → PAUSING	작업자 요청 또는 복구 가능한 장치 오류. pause_reason을 함께 저장
SYS-06 · 07	PAUSING → PAUSED / FAULT_STOP	양쪽 STOP_CONV 확인 시 PAUSED, 정지 확인 실패 시 FAULT_STOP
SYS-08	PAUSED → RUN_SYS	위치·센서·FIFO 정합성 확인. 자동 재개 여부는 pause_reason이 결정
SYS-10 · 15	ANY → FAULT_STOP	추적 무결성 상실, 중대 장비 오류, 물리 E-stop 작동
SYS-11 ~ 13	FAULT_STOP → RESETTING → 복구	guard 충족 시 빈 라인은 READY, 위치 보존 복구는 PAUSED로 복구
SYS-14	ANY → 프로그램 종료	Ctrl+C 포함. 안전 정지를 확인하지 못하면 프로세스를 종료하지 않는다

정지 상태에서의 실패는 재시도를 우선한다. 초기화·리셋·재개 실패는 안전 정지가 확인되는 한 **FAULT_STOP**으로 승격하지 않고 간격 재시도한다. 움직임을 제어할 수 없거나 제품 추적을 신뢰할 수 없을 때만 **FAULT_STOP**을 사용한다. 촬영용 **CAPTURE_HOLD**는 시스템 **PAUSED**가 아니라 **RUN_SYS** 내부의 컨베이어 지역 상태다.

4 일시정지 사유와 재개 방식

PAUSED에서 자동으로 재개할지 작업자를 기다릴지는 pause_reason이 결정한다

pause_reason	발생 상황	재개 방식
OPERATOR	작업자 PAUSE_REQUEST	작업자 RESUME_REQUEST
DEVICE_RECOVERY_AUTO	통신 재연결·소프트 재초기화 등 물리 접촉이 필요 없는 복구	정합성 확인 후 자동 재개
DEVICE_RECOVERY_MANUAL	카메라 반복 실패, 액추에이터 장치 오류 등 물리 점검이 필요한 복구	작업자 RESUME_REQUEST 필수
FAULT_RECOVERY	FAULT_STOP 이후 제품 위치를 보존한 채 복구 완료	작업자 RESUME_REQUEST 필수
STORAGE_RECOVERY	로그 spool이 위험 한도에 도달	저장소 복구 후 작업자 재개

PAUSING 구간의 이벤트 처리

구분	PAUSING 중 처리
센서 1·2·3 확정 이벤트	폐기하지 않고 반영한다. 감속 중 제품이 센서를 통과할 수 있으므로 무시하면 FIFO가 어긋난다
제품 등록 · 판정 잠금	허용. 제품 상태 전이는 RUN_SYS 와 PAUSING 에서 모두 성립한다
ACTUATOR_SCHEDULE	즉시 허용 . route 확보 명령이므로 보류하면 감속 중 제품이 분기점을 미분류로 통과한다
위치 이동 · 재가동 명령	보류 . POSITION_FOR_CAPTURE와 RESUME_AFTER_CAPTURE는 RESUME 이후에 수행한다
진행 중 촬영 · 추론	컨베이어를 움직이지 않는 범위에서 계속 수행하고 결과를 정상 반영한다

작업자가 장비에 손을 대야 하는 복구는 자동 재개하지 않는다. 자동 재개는 소프트웨어만으로 끝나는 복구에 한정하며, E-stop 이력이 있으면 어떤 경우에도 수동 RESET과 START를 요구한다.

5 제품 물리 FSM과 비전 비동기 관계

물리 진행 상태와 촬영·추론 상태를 하나의 선형 상태로 합치지 않는다

제품 물리 상태	physical_zone	의미
STATION_A_WAIT	UPPER_INSPECTION	A 촬영 요청과 응답 대기
STATION_A_DONE	UPPER_INSPECTION	A 촬영 처리 완료, 상층 재가동 대기
FLIPPING	FLIP_TRANSFER	하층으로 이동하며 뒤집히는 구간
STATION_B_WAIT	LOWER_INSPECTION	B 촬영 요청과 응답 대기
STATION_B_DONE	LOWER_INSPECTION	B 촬영 처리 완료, 하층 재가동 대기
SENSOR3_WAIT	LOWER_TO_SENSOR3	분류 센서 도착 대기
AT_SENSOR3	LOWER_TO_SENSOR3	판정과 action 잠금, 액추에이터 위치 추적
ACTUATING	LOWER_TO_SENSOR3	분류 통과 확인 대기
DONE	—	완료 확정, 활성 FIFO에서 제거

비전 작업의 병행 관계

촬영 FSM	추론 FSM	제품 물리 상태와의 관계
IDLE → REQUESTED → CAPTURING → IMAGE_READY / FAILED	QUEUED → RUNNING → DONE / FAILED → ARCHIVED	촬영 응답이 오면 제품은 즉시 다음 물리 상태로 이동하고, 추론은 그 뒤에서 계속 된다

result_locked=true가 된 이후 도착한 결과는 판정과 액추에이터 action을 바꾸지 않는다. 값만 다르면 **LATE_IGNORED**, 제품 연결이 충돌하면 **LATE_CONFLICT_IGNORED**로 기록한다. 촬영은 최초 + 재촬영 1회, 추론은 최초 + 재추론 1회까지만 시도하고 그 이후는 **FORCED_NG**로 확정한다.

6 액추에이터와 FIFO 완료 처리

판정과 FIFO는 Master가, 실행 예약과 route는 Control이 소유한다

Sensor 3 수락

Master가 PASS/REJECT를 잠그고 actuator_job_id와 command_id를 발급한다



Control 예약 큐

bounded 최대 4개. target_passage_step은 Control이 설정 offset으로 계산한다



WAITING_POSITION

같은 action이 연속되면 분기 암을 그대로 유지하고 전환하지 않는다



ACTUATING

예약 route로 제품이 분류 구간을 벗어나는지 스텝 기준으로 감시한다



CONFIRMED

completion_id 생성. Master ACK 전까지 결과를 보존하고 재전송한다



Master 완료 반영

FIFO 선두 여부와 무관하게 actuator_job_id로 직접 매칭한다



FIFO 순차 제거

선두부터 완료 플래그가 연속된 제품만 제거하고 completion_id별로 ACK한다

예약이 여러 개여도 물리 순서를 건너뛰어 제거하지 않는다. 뒤 제품의 완료 이벤트가 먼저 처리되면 완료 표시만 보존하고, 앞 제품이 제거된 뒤 차례대로 제거한다. 도착 전에 route를 확보할 수 없을 때만 ACTUATOR_BUSY로 하층을 정지하며, route 여유 스텝은 반드시 컨베이어 정지 거리보다 크게 설정한다.

7 오류 처리 원칙

오류의 이름이 아니라 제품 추적을 신뢰할 수 있는지로 등급을 나눈다

등급	대표 사례	처리
제품 단위	촬영·추론 최종 실패, Sensor 3 결과 미완료, 동일 제품의 비전 결과 충돌	FORCED_NG 후 계속 운전. 반복 장치 실패만 PAUSED 복구로 승격
경고·저장	이미지 저장 실패, SQLite 쓰기 실패, LogNode 단절	DEGRADED로 계속. spool 위험 한도에서만 PAUSED. FAULT_STOP 없음
복구 가능 장치	Vision 통신 단절, 카메라 반복 실패, 동작 전 액추에이터 오류	PAUSING → PAUSED. 소프트 복구만 자동 재개
추적 무결성	FIFO 불일치, 제품 위치 불명, 명령·이벤트 내용 충돌, route 불명	FAULT_STOP → LINE_CLEAR_REQUIRED → RESETTING
물리 E-stop	빨간 래칭 스위치 작동	구동전원 차단 → FAULT_STOP. 버튼 해제 후 수동 RESET
시작 차단	노드 초기화 실패, Mega 부팅 실패, 인터페이스 버전 불일치	장비가 움직이기 전에 검출. INITIALIZING을 유지하며 재시도

LINE_CLEAR_REQUIRED 해제에는 CLI 확인만으로 부족하다. 상·하층 STOP_CONV, Sensor 1/2/3 CLEAR, 액추에이터 안전 상태와 주변 제품 없음이 모두 충족되어야 RESETTING으로 진입한다. 다만 센서는 점 검출이므로 센서 사이 구간에 남은 제품은 소프트웨어가 확인할 수 없고, 최종적으로는 작업자 확인에 의존한다는 한계가 있다.

8 PoC 물리 비상정지

소프트웨어 경로와 독립적으로 동작하는 유일한 정지 수단

빨간 래칭형 비상정지 스위치

누르면 고정되고 작업자가 돌려서 해제한다. 모터용 전원공급기의 AC 입력에 직렬 설치



TB6600 구동전원 일괄 차단

상층·하층·액추에이터 드라이버 3대가 동시에 전원을 잃는다. PC·Mega·센서·카메라·조명 전원은 유지



PUL 중단 · ENA 비활성 요청

ENA는 정상 운전용 출력 비활성화 신호일 뿐 비상정지 수단으로 신뢰하지 않는다



ESTOP_ASSERTED 보고

Mega가 구동전원 유무를 옵토커플러 입력으로 감시하고 ControlNode가 Master에 보고한다



Master FAULT_STOP

신규 명령 차단. 최종 스텝·FIFO·활성 job 스냅샷을 보존한다



버튼 해제 → 수동 RESET → START

자동 재가동 금지. 전원이 복구되어도 명시적 시작 명령 전까지 움직이지 않는다

전원 차단 방식은 스텝모터의 유지 토크도 함께 제거한다. 수평 컨베이어와 수평 회전 분기 암에서는 문제가 없지만, 중력으로 움직이는 축이 추가되면 기계식 브레이크를 별도로 검토해야 한다. PoC 구성은 정식 안전 인증 회로가 아니라 개발용 긴급 차단 장치이며, 상시 근접 작업이 발생하는 최종 장비에서는 안전 릴레이와 접촉기 구성을 다시 판단한다. 220V 배선 작업은 전기 담당자의 확인을 받는다.

9 구현 착수 전 남은 항목

정책은 확정되었고 남은 것은 실측값과 두 개의 구조 정의

코드 작성 전에 알아야 할 항목

항목	내용	차단 범위
CameraResult 구조체	StationResult.camera_results가 참조하지만 05에 필드 정의가 없다	Vision 결과 직렬화
NodeRuntimeEnvironment	DS-027이 등록되었으나 필드 목록이 비어 있다	초기화 버전 검증
sim profile 값	TODO(HARDWARE_REQUIRED) 29개 항목의 시뮬레이터 초기값이 비어 있다	장비 도착 전 개발 전반

장비 도착 후 확정할 값

구분	확정된 원칙	장비 후 실측
컨베이어·센서	정지 촬영, 시간과 스텝 이중 재감지 조건, 센서 stuck은 FAULT_STOP	steps/mm, 속도, 위치 offset, debounce, 정지거리
카메라·조명	Vision ARM·수집, Control/Mega 하드웨어 트리거와 조명 제어	트리거 전압·펄스폭, 조명 채널·안정화 시간, 3+1 배치
액추에이터	2위치 분기 암, 예약 4개, BUSY 시 하층 정지, 위치 불명은 FAULT_STOP	전환시간, 목표거리, 통과 확인 스텝, 정지 여유
E-stop	TB6600 구동전원 물리 차단, 해제 후 수동 RESET	전원공급기 정격, 스위치·배선, Mega 피드백 핀
실행 환경	Ubuntu 24.04 · ROS 2 Jazzy · Python 3.12	Hikrobot MVS, GPU 드라이버, CUDA, PyTorch 버전

구현 순서 — ① **inspection_interfaces**의 msg·srv·action 정의 → ② 05의 Enum과 dataclass 생성 → ③ 노드별 상태머신 골격과 전이표 단위 시험 → ④ sim profile 기반 정상·오류 시나리오 시험 → ⑤ 실물 연결과 HARDWARE_REQUIRED 값 확정. hardware profile에서 필수값이 비어 있으면 INITIALIZING을 유지하고 장비를 움직이지 않는다.